Rust – Sintaxis del lenguaje

Comentarios

Símbolos	Explicación
`//`	Comentario de línea
`//!`	Comentario de línea de documentación interna
`///`	Comentario de línea de documentación externa
`/*...*/`	Comentario de bloque
`/*!...*/`	Comentario de bloque de documentación interna
`/**...*/`	Comentario de bloque de documentación externa

atajo comentario VS Code → **ctrl + ç**

Operadores básicos

- **Aritméticos**

OPERADOR	DESCRIPCIÓN
+	Suma
–	Resta
*	Multiplicación
/	División (mismo tipo devuelto) x = 4 / 2 o x = 4.0 / 2.0
%	Resto de una división entre enteros

- **Relacionales y lógicos**

OPERADOR	DESCRIPCIÓN
= =	Es igual
!=	Es distinto
<, <=, >, >=	Menor, menor o igual, mayor, mayor o igual
&&	Operador and (y)
||	Operador or (o)
!	Operador not (no)

- **Operadores de asignación** simple y compleja: **= += -= *= /= %=**
- **Puntero**: ***
- **Préstamo**: **&** (&expr, &mut expr, &type, &mut type)

Salida estándar formateada

Rust proporciona una serie de macros definidos en ***std::fmt*** (no es necesario importar) para imprimir en pantalla que es la salida estándar.

- **`format!:`** escribir texto formateado a **`String`**
- **`print!:`** lo mismo que `format!`Pero el texto está impreso en la consola (io::stdout).
- **`println!`**: lo mismo que `print!`Pero se adjunta una nueva línea.
- **`eprint!`**: lo mismo que `print!`Pero el texto se imprime en el error estándar (io::sterr).
- **`eprintln!`**: lo mismo que `eprint!`Pero se adjunta una nueva línea.

```
println!("Hello, world!");     let txt =format!("hola Rust");
```

Variables (en Rust también se llaman enlaces "binding")

Una variable es un contenedor que almacena un valor o información en la memoria. Estos valores pueden ser números, texto, lógicos, u otros tipos de información.

Para reservar el espacio se utiliza la palabra reservada **let** seguida de un **identificador** y una **asignación obligatoria**

`let mi_numero = 16;` → aparecerá automáticamente: i32 que escribe la extensión *rust-analyzer* indicando el tipo de dato, pero no forma parte del código.

Visualización de variables en consola

Podemos visualizar variables mediante salida estándar:

⇒ Colocando **{}** en la posición que ocuparan los contenidos de las variables.

```
    let mi_numero = 16;
    println!("mi_numero = {}", mi_numero);
    let nombre = "Jorgito";
    println!("¡Hola, {}!", nombre);
```

⇒ mediante **interpolación**, poniendo el nombre de la variable dentro de las llaves

```
    let nombre = "Jorgito";
    let edad: u32 = 30;
    println!("Hola {nombre}, tienes {edad} años");
```

⇒ escribiendo el número de argumento dentro de las llaves. Esto es cómodo, por ejemplo, si tienes que repetir el valor

```
println!("Hola {0} tienes {1} años y tu numero es {2}", nombre, edad, mi_numero);
```

** se pueden mezclan los métodos.

```
    let manzanas = 50;
    let naranjas = 14 + 6;
    let frutas = manzanas + naranjas;
    println!(
        "Este año, mi jardín tiene {manzanas} manzanas y {} naranjas.", naranjas
    );
```

En este ejemplo, ten en cuenta que frutas se calcula, pero no se usa, así que Rust mostrará una advertencia de variable sin usar y ofrece como solución ocultarla con un guion bajo:

```
warning: unused variable: `frutas`
  --> src\bin\interpolation_with_curly_braces.rs:12:9
   |
12 |     let frutas = manzanas + naranjas;
   |         ^^^^^^ help: if this is intentional, prefix it with an underscore: `_frutas`
```

Variables mutables e inmutables

Las variables en Rust son **inmutables,** esto es, que no se pueden cambiar de valor en todo el programa.

Si después de declarar y asignar valor a una variable, lo cambias, Rust mostrará un error.

```
let precio: i32 = 100;
println!("El precio es: {precio}");
precio = 120;    cannot assign twice to immutable variable `precio`⏎cannot assign twice to immutable variable
```

```
error[E0384]: cannot assign twice to immutable variable `precio`
 --> src\bin\03_inmutable_mutable_variable.rs:4:5
  |
2 |     let precio = 100;
  |         ------ first assignment to `precio`
3 |     println!("El precio es: {precio}");
4 |     precio = 120;
  |     ^^^^^^^^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
2 |     let mut precio = 100;
```

Los errores de Rust se pueden consultar en la página: https://doc.rust-lang.org/error_codes/error-index.html

Si se permite volver a declarar la misma variable con otro valor, pero desaparece la primera. Se dice que la primera variable queda ***sombreada*** **(Shadowing)** por la segunda.

```
    let precio = 100;
    println!("El precio es: {precio}");
    let precio = 120;
    println!("El precio actualizado es: {precio}");
```

El sombreado u ocultación depende del ámbito (scope)

```
// Shadowing
    let x = 5;
    let x = x + 1;
    {
        let x = x * 2;
        println!("El valor de x en el ámbito interno es: {x}");
    }
    println!("El valor de x es: {x}");
```

Al redefinir una variable también se puede asignar tipos de datos distintos, por ejemplo, primero una cadena y luego un entero.

```
    let spaces = "   ";
    let spaces = spaces.len();
    println!("el valor de spaces es: {spaces}");
```

Esto puede ser útil, por ejemplo, cuando leemos un dato en forma de texto y lo queremos pasar a numérico en la misma variable.

```
    //cambio de tipo de dato
    let total = "23.35";
    let total = 23.34;
    println!("el valor de total es: {total}");
```

Si quieres cambiar el valor de la "**variable**" en el programa tienes que declararla con la palabra **mut**

```
let mut precio_mutable = 100;
println!("El precio mutable es: {precio_mutable}");
precio_mutable = 120;
println!("El precio mutable actualizado es: {precio_mutable}");
```

Las variables mutables **pueden cambiar de valor, pero no de tipo**. En el ejemplo anterior, cambiamos de tipo y luego la redefinimos como mutable.

```
let mut total = 23;
```

Ámbito de variable

El ámbito determina donde se puede utilizar una variable.

En Rust el ámbito viene determinado por el bloque en el que se defina la variable.

```
fn main() {
    let precioo_cafe = 5.99;

    {
        let precio_cafe = 1.99;
        println!("El precio del cafe es: {precio_cafe}");
    }

    println!("El precio del cafe es: {precioo_cafe}");
}
```

Cuando el bloque finaliza la variable es eliminada.

Constantes

Las constantes son valores que no se pueden cambiar asociados a un nombre en mayúsculas.

Las constantes siempre son inmutables y deben tener un tipo definido.

Las constantes son válidas durante todo el tiempo que se ejecuta un programa, dentro del ámbito en el que se declararon.

```
    const APELLIDO : &str = "Donnal";
    const VALOR :i32 = 100;
```

Alias en los tipos de variable

Se puede poner un alias a los tipos para dar claridad al código

```
type Metros = u32;
fn main() {
    let longitud_carrera_milla: Metros = 1600;
    let longitud_carrera_dos_millas: Metros = 3200;
    println!(
        "Una carrera de una milla mide {longitud_carrera_milla} metros y una carrera de
dos millas mide {longitud_carrera_dos_millas} metros."    );
}
```

Directivas de compilación

Es una anotación que añadimos al código que le indica al compilador como analizar el código. Se pueden aplicar directivas a líneas individuales, a funciones o a archivos completos.

Ejemplo: Colocar la directiva que evita warning cuando no se utiliza una variable.

```
#![allow(unused_variables)] //afecta a todo el archivo
#[allow(unused_variables)] //afecta a la función main completa
fn main(){
    #[allow(unused_variables)] // solo afecta a la línea siguiente
    let valor1 = 10;
}
```